Workpaper: core idea and data model

Okafor-Lindqvist 2025 return · live demo https://miraflores-workpaper-seven.vercel.app · long version at `/about`

THE CORE IDEA

A return is a list of **claims**, and the workpaper's job is to clear every one. A claim is anything that must be true for the return to be right: a number on a form, a household fact, a document that must be accounted for, a client answer, an engine warning, one of the firm's twelve required steps. Each keeps the id the pipeline gave it, so a URL, a note, a check result and a sign-off all point at the same thing.

A spreadsheet stores results. A workpaper has to prove why each result is right, and that proof is relational: a number connects to its source page, to how it was built, to who checked it, to the client message that changed it, and to last year. So I store the claims and an append-only log of what people did to them, and compute everything else from those two.

THE DATA MODEL

1. Packet (sealed)

74 documents, 3,012 extracted values, 3 return versions. SHA-256 pinned, never written. SSNs, bank numbers and IP PINs are masked into tokens that still compare equal before they leave the server.

→

2. Claims and checks

818 claims: 627 return lines, 74 documents, 56 client answers, messages and calls, 22 household facts, 13 engine warnings, 12 firm steps, 14 other. 38 engine-free checks attach 152 findings.

→

3. Acts (stored)

The only stored state. Append-only Postgres rows, each hash-chained to the one before; a trigger refuses update, delete and truncate. 17 Zod-validated kinds, plus 2 the server writes.

→

4. State (derived)

`project(model, acts, viewer)` gives every screen its state: effective values, tax effects, open items, sign-offs, gates. Browser and server run the same code.

An act is one row: `seq, id, engagement, recorded_at, season_at, actor, actor_role, type, target, refs[], body, derived, sealed, batch_id, prev_hash, hash`. `target` and `refs` use the packet's own ids (`F8949.B.N1`, `D28`, `F1040.26`). `derived` (tax effect, partner routing, sign-off fingerprints) is computed only by the server. `sealed` holds clear values, such as a corrected SSN, that never reach a browser.

What follows from that shape:

- **Nothing is overwritten.** A claim keeps its packet value; its effective value is the latest live adjust act on it, so who changed it, why and the tax effect stay on the act. Undo is a `revert` act under the same rules.
- **Sign-offs are data, not flags.** One stores a fingerprint of the values and evidence it covered and is valid only while that still matches, so it reopens by itself.
- **Two clocks.** `recorded_at` is audit truth. `season_at` replays the season, so documents and messages appear when the season clock reaches them.
- **Tax effect is change × the packet's rates,** or the exact `what_if` result where one exists, because the engine cannot be rerun. Every total is labelled an estimate.

WHAT THE SERVER ENFORCES

The same `authorize()` runs in the browser, to explain a refusal, and in the server's write transaction, to make it. No one reviews work they prepared, packet history included. A change over 1,000 needs a valid sign-off from a partner who did not prepare it, judged on the running total. Open blocking review notes stop approval and filing. Client approval binds the exact amounts. Filing freezes the return; later changes are amendments. Every reveal of a masked value, granted or refused, is logged first.

ERRORS, LIMITS AND HOW AI WAS USED

Errors. `/errors` lists all 70 issues I found, each with its check or marked as reviewer judgment.

Limits. The role switcher is a demo identity picker, not authentication: a signed cookie, with role, clearance and start date re-resolved on the server for every request. Totals are linear estimates, e-file is one disclosed rule, e-signature is a PIN, notifications are in-app only.

AI. I built it with Claude Code and made the calls (scope, the masking model, the partner rule, what to fake). A script compared all 3,012 extracted values with their printed pages; ten analyst agents (one per return area), an adversarial verifier per area and a completeness critic then produced the 70 issues. Judge agents scored competing designs; parallel build agents owned slices behind frozen contracts, each reviewed independently; every check is pinned by a test against the real packet.

Page 1 of the brief PDF is blank in my copy, so for what it would have carried (Evidence, Calculation) I worked from the task text.